

```
int somma(int a, int b)
{
    if (a == 0) return b;
    return somma(a - 1, b + 1);
}
```

Figure 1: Esempio di pseudocodice

```
int somma(int a, int b)
{
    if (a == 0) return b;
    return somma(a - 1, b) + 1;
}
```

Figure 2: Esempio di pseudocodice

Rispondere alle domande a risposta multipla annerendo la casella corrispondente alla risposta corretta. Ogni domanda ha una ed una sola risposta corretta.

Cognome e Nome:

Matricola:

Domanda 1 Funzione implementata dallo pseudo-codice di Figura 1:

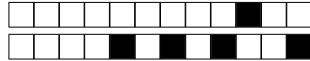
- ☐ Causa sempre ricorsione infinita
- ☐ Usa ricorsione in coda
- ☐ Nessuna delle altre risposte
- ☐ Non usa ricorsione in coda
- ☐ Non può essere implementata per via iterativa

Domanda 2 Funzione implementata dallo pseudo-codice di Figura 2:

- ☐ Non può essere implementata per via iterativa
- ☐ Usa ricorsione in coda
- ☐ Causa sempre ricorsione infinita
- ☐ Nessuna delle altre risposte
- ☐ Non usa ricorsione in coda

Domanda 3 β -riducendo $((\lambda a.a)(\lambda b.bb))(\lambda c.cd)$ si ottiene:

- ☐ dd
- ☐ La riduzione non termina
- ☐ y
- ☐ $\lambda x.xa$
- ☐ Nessuna delle altre risposte



```
int b = 666;

int pippo(int x)
{
    x = 666; b = 1;
    return x / 2;
}

int pluto(void)
{
    int a, c;

    a = b / 333;
    c = pippo(a);

    return a + c;
}
```

Figure 3: Esempio di pseudocodice

Domanda 4 La memoria gestita dinamicamente:

- ☐ E' necessaria per implementare l'iterazione
- ☐ Non è mai strettamente necessaria, ma permette di ottenere migliori prestazioni
- ☐ E' usata solo in linguaggi interpretati
- ☐ E' necessaria per implementare la ricorsione
- ☐ Nessuna delle altre risposte

Domanda 5 Se gli array sono memorizzati *per righe* e `char a[100][100]` è un array multidimensionale di caratteri con `a[0][0]` che ha indirizzo `0x1100`, qual'è l'indirizzo di `a[5][10]`?:

- ☐ `0x12FE`
- ☐ `0x1510`
- ☐ `0x22FE`
- ☐ Nessuna delle altre risposte
- ☐ `0x210F`

Domanda 6 Si consideri lo pseudo-codice di Figura 3. Qual'è il valore di ritorno di `pluto()` se i parametri sono passati *per riferimento*?

- ☐ Nessuna delle altre risposte
- ☐ 999
- ☐ Non è possibile passare `a` per riferimento (perché `a = b / 333`)
- ☐ 333
- ☐ 666



```
int b = 666;

int pippo(int x)
{
    x = 666; b = 1;
    return x / 2;
}

int pluto(void)
{
    int a, c;

    a = b / 333;
    c = pippo(a);

    return a + c;
}
```

Figure 4: Esempio di pseudocodice

Domanda 7 Considerando la macchina astratta che permette di eseguire un linguaggio $\mathcal{L}$, il numero di record di attivazione contemporaneamente presenti nel sistema:

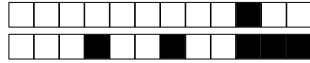
- ☐ Nessuna delle altre risposte
- ☐ Può avere un massimo noto a priori solo se il linguaggio $\mathcal{L}$ permette ricorsione
- ☐ Non dipende dal programma che si esegue
- ☐ Ha un massimo noto a priori solo se la macchina astratta alloca i record di attivazione dallo heap
- ☐ E' sempre determinabile a priori da un'analisi statica del codice del programma eseguito

Domanda 8 Si consideri lo pseudo-codice di Figura 4. Qual'è il valore di ritorno di `pluto()` se i parametri sono passati *per valore*?

- ☐ Nessuna delle altre risposte
- ☐ 3
- ☐ 0
- ☐ Dipende dal tipo di scope (statico o dinamico) utilizzato
- ☐ 1

Domanda 9 Dato il frammento di programma (espresso in pseudo-codice) della Figura 5, quanto vale la variabile globale `c` dopo aver eseguito `topolino()`, assumendo scope dinamico?

- ☐ Nessuna delle altre risposte
- ☐ Non è possibile dirlo
- ☐ 14
- ☐ 6
- ☐ 3



```
int a, b, c;

void pippo(void)
{
    int a;

    a = 6;
    b = 5;
}

void pluto(void)
{
    int c;
    int b;

    pippo();
    c = 3;
    a = 4;
}

void topolino(void)
{
    int a;

    a = 1;
    b = 10;
    pluto();

    c = a + b;
}
```

Figure 5: Esempio di pseudocodice



```
int a, b, c;

void pippo(void)
{
    int a;

    a = 6;
    b = 5;
}

void pluto(void)
{
    int c;
    int b;

    pippo();
    c = 3;
    a = 4;
}

void topolino(void)
{
    int a;

    a = 1;
    b = 10;
    pluto();

    c = a + b;
}
```

Figure 6: Esempio di pseudocodice

Domanda 10 Un *compilatore* da un linguaggio $\mathcal{L}$ ad un linguaggio $\mathcal{L}_O$ è:

- ☐ Una implementazione di macchine astratte indipendente dalla macchina fisica
- ☐ Nessuna delle altre risposte
- ☐ L'implementazione di una macchina astratta scritta nel linguaggio $\mathcal{L}_O$, che capisce programmi scritti nel linguaggio $\mathcal{L}$
- ☐ Un programma che trasforma un programma $P^{\mathcal{L}_O}$ (espresso nel linguaggio $\mathcal{L}_O$) in un programma $P^{\mathcal{L}}$ (espresso nel linguaggio $\mathcal{L}$) tale che per ogni input I si ha $P^{\mathcal{L}}(I) = P^{\mathcal{L}_O}(I)$
- ☐ Un programma scritto nel linguaggio $\mathcal{L}_O$ che riceve come ingresso un programma $P^{\mathcal{L}}$ (espresso nel linguaggio $\mathcal{L}$) ed il suo input I generando lo stesso output che genera $P^{\mathcal{L}}$ con input I

Domanda 11 Dato il frammento di programma (espresso in pseudo-codice) contenuto in Figura 6, quanto vale la variabile globale `c` dopo aver eseguito `topolino()`, assumendo scope statico?

- ☐ Nessuna delle altre risposte
- ☐ Non è possibile dirlo
- ☐ 6
- ☐ 3
- ☐ 14



Domanda 12 β -riducendo $(\lambda a.((a\lambda b.\lambda c.c)\lambda d.\lambda e.d))(\lambda f.\lambda g.f)$ si ottiene:

- ☐ c
- ☐ $\lambda b.\lambda c.c$
- ☐ $\lambda b.\lambda c.b$
- ☐ La riduzione non termina
- ☐ Nessuna delle altre risposte